KUBERNETES-03

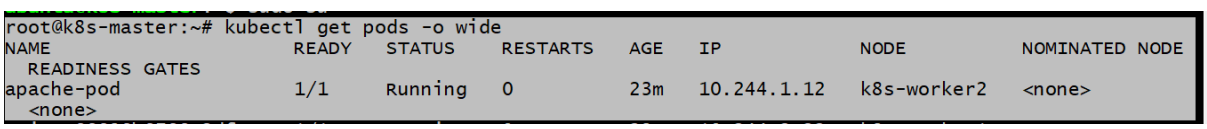
Created and Test a ClusterIP Service for an Apache Web Server

1. Objective

Create a Kubernetes ClusterIP Service for an Apache HTTPD pod and verify that other pods can access Apache using the Service DNS name.

- Deployed an Apache HTTPD pod using the official `httpd` Docker image.
- Verified the Apache pod status and assigned Pod IP.
- Created a ClusterIP Service named `apache-service`.
- Configured Service port 80 and target port 80.
- Tested Apache using the Kubernetes Service DNS name.
- Confirmed successful HTTP response from the Apache web server.

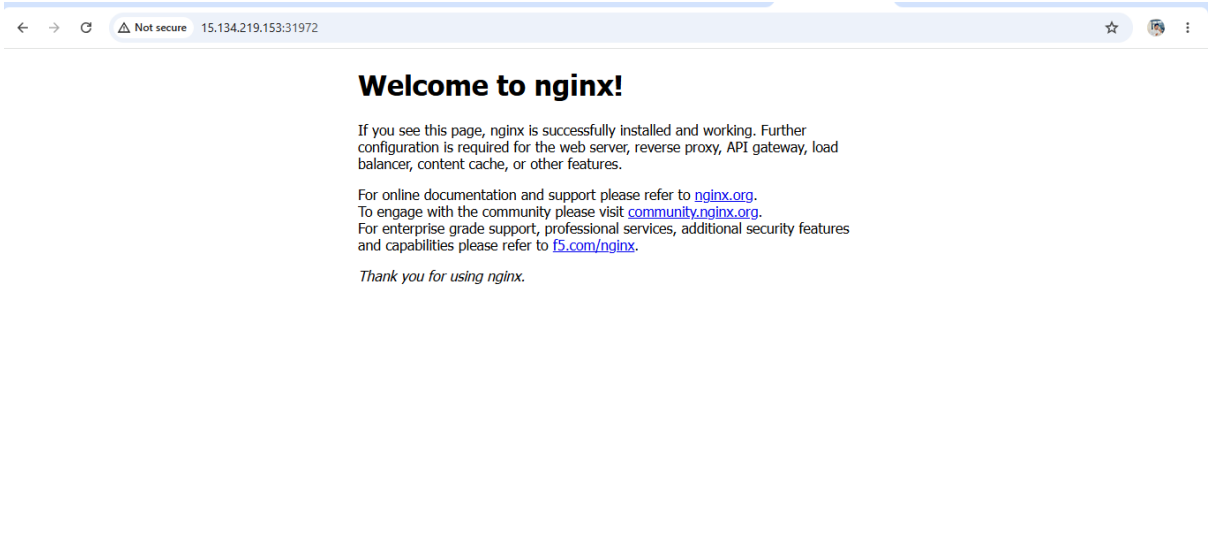




Expose an Nginx Pod Externally Using a NodePort Service

- Assigned the label `app=nginx-nodeport` to the pod.
- Exposed the Nginx pod using a NodePort Service.
- Configured Service port 80 and target port 80.
- Verified the assigned ClusterIP and NodePort.
- Added an inbound TCP rule for port 31972 in the AWS Security Group, restricted to the client IP.
- Accessed the Nginx web server through the EC2 public IP and NodePort.
- Verified the default Nginx welcome page in the browser.

```
root@k8s-master:~# kubectl get svc nginx-nodeport-service
NAME                    TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
nginx-nodeport-service   NodePort   10.110.21.48   <none>        80:31972/TCP   10s
```



Deployed ReplicationController to Maintain 3 Replicas of an Nginx Pod

- Created a Kubernetes ReplicationController YAML manifest.
- Configured the desired replica count to three.
- Defined the Nginx container using the official `nginx:latest` image.

- Applied the YAML manifest to deploy the ReplicationController.
- Verified the creation of three Nginx pod replicas.
- Tested automatic pod replacement by deleting one replica.
- Confirmed that the ReplicationController maintained the desired replica count.

```
root@k8s-master:~# vi nginx-replicationcontroller.yaml
root@k8s-master:~# kubectl apply -f nginx-replicationcontroller.yaml
replicationcontroller/nginx-rc created
root@k8s-master:~# kubectl get pods
```

```
nginx-rc-2h9xq          1/1     Running   0          93s
nginx-rc-t6dgn          1/1     Running   0          93s
nginx-rc-v2k6v          1/1     Running   0          93s
```

ReplicationController from 3 to 5 Replicas

1. Verified the existing ReplicationController with three replicas.
2. Updated the desired replica count from three to five.
3. Applied the updated YAML configuration.
4. Kubernetes automatically created two additional Nginx pods.
5. Verified five running replicas using `kubectl get pods`.
6. Confirmed the desired and current replica counts using `kubectl get rc`.

```
root@k8s-master:~# kubectl get rc nginx-rc
NAME       DESIRED   CURRENT   READY   AGE
nginx-rc   3         3         3       5m37s
root@k8s-master:~# vi nginx-replicationcontroller.yaml
root@k8s-master:~# kubectl apply -f nginx-replicationcontroller.yaml
replicationcontroller/nginx-rc configured
root@k8s-master:~# kubectl get rc nginx-rc
NAME       DESIRED   CURRENT   READY   AGE
nginx-rc   5         5         5       6m56s
root@k8s-master:~# kubectl get pods
```

```
nginx-rc-8mmf7              1/1     Running   0          19s
nginx-rc-dnt4f              1/1     Running   0          19s
nginx-rc-t6dgn              1/1     Running   0          7m4s
nginx-rc-v2k6v              1/1     Running   0          7m4s
nginx-rc-wdmcw              1/1     Running   0          3m54s
```

```
nginx-rc-8mmf7          1/1     Running   0          60s     10.244.2.28   k8s-worker1   <none>
      <none>
nginx-rc-dnt4f          1/1     Running   0          60s     10.244.1.23   k8s-worker2   <none>
      <none>
nginx-rc-t6dgn          1/1     Running   0          7m45s   10.244.1.21   k8s-worker2   <none>
      <none>
nginx-rc-v2k6v          1/1     Running   0          7m45s   10.244.1.19   k8s-worker2   <none>
      <none>
nginx-rc-wdmcw          1/1     Running   0          4m35s   10.244.1.22   k8s-worker2   <none>
      <none>
```

```
root@k8s-master:~# kubectl get rc nginx-rc
NAME       DESIRED   CURRENT   READY   AGE
nginx-rc   5         5         5       10m
```

Created a ReplicaSet to Manage Pods Using Multiple Labels (prod and test)

```
root@k8s-master:~# kubectl describe rs nginx-replicaset
Name:         nginx-replicaset
Namespace:    default
Selector:     app=nginx,environment in (prod,test)
Labels:       <none>
Annotations:  <none>
Replicas:     3 current / 3 desired
Pods Status:  3 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=nginx
           environment=prod
  Containers:
   nginx:
    Image:         nginx:latest
    Port:          80/TCP
    Host Port:     0/TCP
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Events:
  Type    Reason            Age    From                   Message
  ----    ------            ----   ----                   -------
  Normal  SuccessfulCreate  2m20s  replicaset-controller  Created pod: nginx-replicaset-96jmw
  Normal  SuccessfulCreate  2m20s  replicaset-controller  Created pod: nginx-replicaset-wtk49
  Normal  SuccessfulCreate  2m20s  replicaset-controller  Created pod: nginx-replicaset-gds6t
root@k8s-master:~#
```

```
replicaset.apps/nginx-replicaset created
root@k8s-master:~# cat nginx-replicationcontroller.yaml


apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
    matchExpressions:
      - key: environment
        operator: In
        values:
          - prod
          - test
  template:
    metadata:
      labels:
        app: nginx
        environment: prod
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

```
nginx-replicaset-96jmw   1/1     Running   0          4m20s   app=nginx,environment=prod
nginx-replicaset-gds6t   1/1     Running   0          4m20s   app=nginx,environment=prod
nginx-replicaset-wtk49   1/1     Running   0          4m20s   app=nginx,environment=prod
root@k8s-master:~#
```

ReplicaSet That Excludes Pods with the Label backend

- Created a ReplicaSet YAML manifest.
- Configured `matchExpressions` for label-based pod selection.
- Used the `In` operator to select pods labelled `app=nginx`.
- Used the `NotIn` operator to exclude pods labelled `tier=backend`.
- Configured three Nginx replicas with the `tier=frontend` label.
- Verified the ReplicaSet and pod labels.

```
nginx-replicaset-notin-5f2sq   1/1     Running   0          102s    app=nginx,tier=frontend
nginx-replicaset-notin-77gdc   1/1     Running   0          102s    app=nginx,tier=frontend
nginx-replicaset-notin-9wk92   1/1     Running   0          102s    app=nginx,tier=frontend
```

```
root@k8s-master:~# kubectl describe rs nginx-replicaset-notin
Name:         nginx-replicaset-notin
Namespace:    default
Selector:     app in (nginx),tier notin (backend)
Labels:       <none>
Annotations:  <none>
Replicas:     3 current / 3 desired
Pods Status:  3 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=nginx
           tier=frontend
  Containers:
   nginx:
    Image:         nginx:latest
    Port:          80/TCP
    Host Port:     0/TCP
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Events:
  Type    Reason            Age    From                   Message
  ----    ------            ----   ----                   -------
  Normal  SuccessfulCreate  2m30s  replicaset-controller  Created pod: nginx-replicaset-notin-77gdc
  Normal  SuccessfulCreate  2m30s  replicaset-controller  Created pod: nginx-replicaset-notin-9wk92
  Normal  SuccessfulCreate  2m30s  replicaset-controller  Created pod: nginx-replicaset-notin-5f2sq
root@k8s-master:~#
```

Test Load Balancing Across Multiple Pods Using a NodePort Service

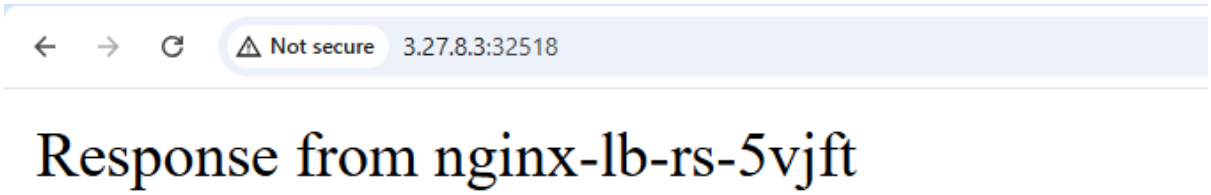
- Created a ReplicaSet with three Nginx pod replicas.
- Configured unique pod labels for load-balancing resources.
- Created a NodePort Service to expose the Nginx pods.
- Verified the Service and its backend endpoints.
- Created a temporary curl pod to generate multiple HTTP requests.
- Verified that requests were served by different Nginx pod replicas.

- Tested external access through the EC2 public IP and NodePort.

```
root@k8s-master:~# vi  nginx-loadbalancer.yaml
root@k8s-master:~# kubectl apply -f nginx-loadbalancer.yaml
replicaset.apps/nginx-lb-rs created
service/nginx-lb-service created
root@k8s-master:~# kubectl get rs nginx-lb-rs
NAME          DESIRED   CURRENT   READY   AGE
nginx-lb-rs   3         3         3       9s
root@k8s-master:~# kubectl get pods -l app=nginx-lb -o wide
NAME                READY   STATUS    RESTARTS   AGE   IP            NODE          NOMINATED NODE
DINESS GATES
nginx-lb-rs-5vjft   1/1     Running   0          18s   10.244.1.27   k8s-worker2   <none>
ne>
nginx-lb-rs-flmgt   1/1     Running   0          18s   10.244.2.32   k8s-worker1   <none>
ne>
nginx-lb-rs-ltgns   1/1     Running   0          18s   10.244.2.33   k8s-worker1   <none>
ne>
root@k8s-master:~# kubectl get svc nginx-lb-service+
Error from server (NotFound): services "nginx-lb-service+" not found
root@k8s-master:~#
root@k8s-master:~# kubectl get svc nginx-lb-service+
Error from server (NotFound): services "nginx-lb-service+" not found
root@k8s-master:~# kubectl get svc nginx-lb-service
NAME               TYPE       CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
nginx-lb-service   NodePort   10.109.161.124   <none>        80:32518/TCP   52s
root@k8s-master:~#
```




```
root@k8s-master:~# kubectl run curl-test \
  --image=curlimages/curl:latest \
  --restart=Never \
  --command -- sleep 3600
pod/curl-test created
root@k8s-master:~# kubectl exec curl-test -- sh -c \
'for i in $(seq 1 10); do curl -s http://nginx-lb-service; done'
Response from nginx-lb-rs-ltgns
Response from nginx-lb-rs-5vjft
Response from nginx-lb-rs-5vjft
Response from nginx-lb-rs-5vjft
Response from nginx-lb-rs-ltgns
Response from nginx-lb-rs-5vjft
Response from nginx-lb-rs-flmgt
Response from nginx-lb-rs-flmgt
```



Deleting a ReplicationController Without Affecting Running Pods

1. Verified the existing Nginx ReplicationController and its managed pods.
2. Deleted the ReplicationController using the `--cascade=orphan` option.
3. Preserved the existing Nginx pods after controller deletion.
4. Verified that the ReplicationController was removed successfully.
5. Confirmed that the orphaned Nginx pods continued running independently.
6. Verified pod ownership to ensure the pods were no longer managed by the deleted controller.

```
root@k8s-master:~# kubectl delete replicationcontroller nginx-rc --cascade=orphan
replicationcontroller "nginx-rc" deleted from default namespace
root@k8s-master:~# kubectl get rc
No resources found in default namespace.
root@k8s-master:~# kubectl get pods -o wide
```

```
NAME                    OWNER                   STATUS
nginx-66686b6766-2dfcx  nginx-66686b6766        Running
nginx-66686b6766-b9zkf  nginx-66686b6766        Running
nginx-66686b6766-lhp9r  nginx-66686b6766        Running
```

Important Kubernetes commands to remember

```
kubectl apply -f
kubectl get pods
kubectl get pods -o wide
kubectl get svc
kubectl get rc
kubectl get rs
kubectl describe pod POD-NAME
kubectl describe rc RC-NAME
kubectl describe rs RS-NAME
kubectl scale rc nginx-rc --replicas=5
kubectl delete rc nginx-rc --cascade=orphan
kubectl get pods --show-labels
```

Concepts covered

- NodePort Services
- ReplicationController
- ReplicaSet
- Scaling replicas
- Labels and selectors
- `matchExpressions`
- `In` and `NotIn` operators
- Service load balancing
- Orphan deletion